

Guión detallado — Defensa MaduraApp

~14 min. Lee “Di” si te bloqueas; usa “Clave” para dar vistazos; “Si profundizan” es para el jurado.

Antes de empezar (en la sala)

- **Start Lab** → consola AWS → **EC2** → **Iniciar instancia** → esperar 2-3 min.
 - Verificar en el navegador del teléfono:
`http://3.215.43.61:8000/v1/health` → debe decir **model_loaded:true**.
 - Tener la app abierta y con sesión iniciada.
-

Slide 1 · Portada (~25s)

DI:

Buenos días. Mi nombre es Claudio Aro, sección cero cero uno D. Voy a defender el Estado de Avance 3 de mi proyecto MaduraApp, un sistema que analiza la madurez de frutas mediante visión computacional. En los próximos quince minutos les mostraré tres cosas: el plan de pruebas que diseñé, cómo lo apliqué a cada componente del sistema, y las mejoras que hice a partir de esos resultados.

CLAVE (vistazo):

- Nombre + sección.
- El objetivo en una frase: plan de pruebas, aplicación y mejoras.

Slide 2 · Agenda (~20s)

DI:

Voy a seguir esta ruta: primero el contexto del problema para situarlos; luego el corazón de esta evaluación, que es el plan de pruebas, su aplicación y los resultados; después las mejoras que apliqué al producto; y cierro con la conclusión y el trabajo pendiente.

CLAVE (vistazo):

- 5 puntos: contexto, plan, aplicación, mejoras, conclusión.

Slide 3 · Contexto del proyecto (~1.5 min)

DI:

Partamos por el problema. En Chile se pierde entre el veinte y el cuarenta por ciento de la fruta después de la cosecha, según la FAO y ODEPA. Una causa raíz es que no existe una forma objetiva y accesible de saber cuándo una fruta está en su punto óptimo de consumo: la gente se guía por el ojo o el tacto, y se equivoca. MaduraApp resuelve esto con una foto: el usuario elige la fruta, toma una foto, y la aplicación le entrega el estado de madurez —inmaduro, óptimo o sobremaduro— con un semáforo de colores y una recomendación. Funciona con cuatro frutas: aguacate, plátano, tomate y mango. Por debajo, la app es Android

nativo en Kotlin, el servidor es una API en FastAPI, y el modelo de inteligencia artificial es YOLO veintiséis, que entrené y que alcanza una precisión, medida en mAP cincuenta, de cero coma noventa y dos, muy por encima del objetivo de cero coma setenta y cinco.

CLAVE (vistazo):

- **20-40%** pérdida post-cosecha (FAO/ODEPA).
- 4 frutas × 3 estados.
- Stack: Android Kotlin + FastAPI + **YOLO26n**.
- **mAP@50 = 0.92** (meta 0.75).

SI PROFUNDIZAN:

Si preguntan por qué YOLO: es un modelo de detección de objetos en tiempo real, liviano (5,2 MB), que corre en CPU sin necesidad de GPU. Lo entrené con un conjunto de imágenes etiquetadas por estado de madurez.

Slide 4 · Plan de pruebas (~2.5 min) [CLAVE]

DI:

Vamos al plan de pruebas, el primer criterio de esta evaluación. Diseñé cincuenta y siete casos de prueba automatizados: treinta y ocho para el backend y diecinueve para la app Android. Y los organicé en cuatro tipos, porque no todas las pruebas verifican lo mismo. Las de validación responden a la pregunta: ¿estoy construyendo el sistema correcto?, es decir, si cumple los requisitos. Las de verificación responden: ¿lo estoy construyendo correctamente?, y miran calidad, como el rendimiento o la precisión. Las de seguridad las basé en OWASP, el estándar de la industria. Y las operacionales verifican que el sistema funcione en su entorno. Un punto importante es la base de datos de pruebas: no uso la base real, uso una base SQLite en memoria que se crea y se destruye en cada corrida; esto aísla cada prueba, es rápido, y no expone datos reales de usuarios.

CLAVE (vistazo):

- **57 casos** = 38 backend + 19 Android.
- Validación=requisitos · Verificación=calidad · Seguridad=OWASP · Operacional=entorno.
- **21 casos** verifican el **estado de madurez** de la fruta (prioridad).
- BD de pruebas: **SQLite en memoria** (aísla, rápido, sin datos reales).

SI PROFUNDIZAN:

Cada caso documenta: funcionalidad, acción o dato de entrada, resultado esperado y obtenido. Ejemplos reales: pedir /predict sin token devuelve 401; un formato de imagen no soportado devuelve 400; una contraseña débil al registrarse devuelve 422.

Slide 5 · Aplicación y resultados (~2 min)

DI:

Estos son los resultados de aplicar el plan. Las cincuenta y siete pruebas pasan al cien por ciento; las ejecuté hoy. El backend con pytest: treinta y ocho de treinta y ocho. La app Android con MockK y JUnit: diecinueve de diecinueve. Además agregué integración continua: configuré un flujo en GitHub Actions que ejecuta toda la suite cada vez que subo un cambio, así me entero de inmediato si algo rompe una prueba y no integro código defectuoso. En calidad: el modelo cumple su objetivo con cero coma noventa y dos, pesa solo cinco coma dos megabytes, y la app compila e instala sin errores.

CLAVE (vistazo):

- **57/57 en verde** (hoy).
- backend 38/38 (pytest) · Android 19/19 (MockK+JUnit).
- **Rendimiento**: ~200 ms inferencia · 0 errores hasta 50 usuarios concurrentes.
- **Integración continua** en cada push.
- Modelo 0.92 · 5,2 MB · APK OK.

SI PROFUNDIZAN:

El backend prueba con pytest sobre SQLite en memoria; Android usa MockK para simular la API y la base Room, y kotlinx-coroutines-test para controlar la concurrencia de forma determinista. El CI es un workflow llamado `backend_ci.yml`.

Slide 6 · Mejoras al producto (~2.5 min) [CLAVE]

DI:

Este es, para mí, el punto más importante. Quiero destacar algo: estas mejoras no son arbitrarias. Cada una nace de un resultado de las pruebas o de una auditoría de seguridad que hice con OWASP. Las mapeé a los cinco estándares de calidad que pide la rúbrica. En seguridad, que fue el foco: descubrí que la clave secreta de los tokens estaba escrita en el código, que la información viajaba sin cifrar, y que el token se guardaba sin cifrar en el teléfono; corregí las tres cosas. En usabilidad, rediseñé toda la interfaz con Material Design tres, con modo oscuro e íconos vectoriales. En corrección, dejé la suite y la compilación en verde tras integrar las funciones nuevas. En completitud, integré y probé de extremo a extremo la autenticación y el sistema de calificación. Y en pertinencia, las recomendaciones están ajustadas al dominio agrícola. Cada mejora tiene un commit que la respalda, así que son trazables.

CLAVE (vistazo):

- **15 mejoras**, todas trazables a un commit.
- 5 estándares: **seguridad, usabilidad, corrección, completitud, pertinencia**.

SI PROFUNDIZAN:

Ejemplos concretos de seguridad: el secreto JWT estaba fijo en el código → ahora va por variable de entorno y la app se niega a arrancar con el valor por defecto en producción. El token se guardaba en texto plano → ahora se cifra en reposo con EncryptedSharedPreferences (AES-256). El tráfico iba en claro → ahora se fuerza HTTPS.

Slide 7 · Seguridad / OWASP (~1.5 min)

DI:

Quiero profundizar un momento en seguridad, porque se trata de proteger los datos personales del usuario. Apliqué OWASP punto por punto: las contraseñas se guardan con un hash de bcrypt, nunca en texto plano. Los endpoints están protegidos con tokens JWT. La comunicación va cifrada por HTTPS, eso es el cifrado en tránsito. Y el token se guarda cifrado en el dispositivo, eso es el cifrado en reposo. Y aquí hay un punto de ética que quiero mencionar: estuve tentado de mostrarle al usuario un mensaje tipo cifrado de extremo a extremo, como WhatsApp. Pero decidí no hacerlo, porque sería falso: mi servidor necesita leer la imagen para procesarla. Sería engañar al usuario. Así que uso un mensaje veraz: tus datos viajan cifrados. La honestidad con el usuario también es parte de la calidad.

CLAVE (vistazo):

- bcrypt (hash) · JWT · HTTPS (tránsito) · AES-256 (reposo).
- Argumento ético: no es “extremo a extremo” porque el servidor lee la imagen.

SI PROFUNDIZAN:

Diferencia clave: el hashing (bcrypt) es irreversible y se usa para contraseñas; el cifrado (AES) es reversible con una clave y se usa para el token. En OWASP: A01 es control de acceso roto, A02 fallos criptográficos, A07 fallos de autenticación.

Slide 8 · Conclusión (~1.5 min) [CLAVE]

DI:

Para concluir. MaduraApp es hoy un producto funcional, probado y seguro. Cincuenta y siete de cincuenta y siete pruebas en verde, quince mejoras trazables a los estándares de calidad, y los datos personales protegidos con cifrado y hashing. De hecho, lo probé en producción: desplegué el backend en una instancia de AWS con Docker, y escaneé un plátano real que diagnosticó como óptimo correctamente. Y soy honesto con lo que queda pendiente: el despliegue, por ser un laboratorio académico, es temporal; falta automatizar las pruebas de extremo a extremo, y obtener su aprobación formal del plan de pruebas en esta misma defensa.

CLAVE (vistazo):

- **57/57 · 15 mejoras** · datos protegidos.
- Evidencia fuerte: **demo real en AWS** (plátano → Óptimo).
- Pendiente honesto: E2E + aprobación del plan.

Slide 9 · Cierre (~10s)

DI:

Eso es todo de mi parte. Muchas gracias por su atención. Quedo atento a sus preguntas.

Glosario técnico (para no trabarte)

- **Validación:** ¿el sistema correcto? (cumple requisitos). **Verificación:** ¿correctamente? (calidad).
 - **Prueba unitaria:** prueba una unidad aislada. **Integración:** varios componentes juntos.
 - **Fixture:** prepara datos antes de un test. **Mock:** objeto que simula una dependencia.
 - **JWT:** token firmado que prueba la identidad en cada petición.
 - **bcrypt / hash:** transformación irreversible; la contraseña nunca se guarda en claro.
 - **AES-256:** cifrado reversible con clave; protege el token en el dispositivo.
 - **HTTPS / TLS:** cifra la comunicación entre app y servidor (cifrado en tránsito).
 - **OWASP:** lista estándar de los 10 riesgos de seguridad más críticos en apps.
 - **mAP@50:** métrica de precisión del modelo (0.92 = muy alta).
 - **CI:** integración continua; corre las pruebas automáticamente en cada cambio.
 - **Endpoint:** una ruta de la API (por ej. /predict, /history).
-

Banco de preguntas (Q&A)

P: ¿Por qué SQLite en memoria para las pruebas?

R: Para aislar cada prueba (que una no contamine a otra), por velocidad y para no exponer datos reales. El esquema es el mismo de producción, así que la prueba es representativa.

P: ¿Por qué simulan (mockean) el modelo YOLO en los tests?

R: Porque el test del endpoint valida la lógica de la API —validación de entrada, persistencia, respuesta—, no la red neuronal. Así es rápido y determinista. La precisión del modelo se mide aparte con la métrica mAP.

P: ¿Cuál es la diferencia entre validación y verificación?

R: Validación responde si construyo el sistema correcto (cumple los requisitos del usuario). Verificación, si lo construyo correctamente (atributos de calidad como rendimiento o precisión).

P: ¿Por qué no es cifrado de extremo a extremo como WhatsApp?

R: Porque en extremo a extremo ni el servidor puede leer el contenido, y mi servidor necesita leer la imagen para procesarla y guardar el historial. Llamarlo así sería falso. Uso el término correcto: cifrado en tránsito más cifrado en reposo.

P: ¿Cómo aseguran que las pruebas se ejecutan siempre?

R: Con integración continua: un workflow de GitHub Actions corre toda la suite en cada push. Si algo rompe una prueba, se detecta antes de integrar.

P: ¿De dónde salieron las mejoras?

R: De los resultados de las pruebas y de una auditoría OWASP. Por ejemplo, al integrar la autenticación, las pruebas quedaron en rojo y las corregí; y la auditoría reveló el secreto JWT en el código, que moví a variable de entorno. Cada mejora tiene su commit.

P: ¿Qué pasa si alguien intercepta la red?

R: No ve nada útil: el tráfico va cifrado por HTTPS, y la contraseña nunca viaja ni se guarda en claro porque se hashea con bcrypt.

P: ¿Cómo guardan la sesión en el teléfono de forma segura?

R: El token JWT se cifra en reposo con EncryptedSharedPreferences (AES-256) y la clave vive en el Android Keystore. Si extraen el archivo del dispositivo, no pueden leer el token.

P: ¿Qué es mAP@50?

R: Mean Average Precision con un umbral de solapamiento de 0,5; mide qué tan bien el modelo detecta y clasifica. 0,92 es muy alta, sobre el objetivo de 0,75.

P: ¿Qué cobertura tienen las pruebas?

R: Los 57 casos cubren autenticación, inferencia, historial, feedback, caché offline y la capa de presentación. Hay una matriz que relaciona cada requisito con las pruebas que lo cubren.

P: ¿Cómo prueban el estado de madurez de la fruta?

R: Con 21 casos que simulan la salida del modelo y verifican que cada detección se traduzca en el estado, el color de semáforo y la recomendación correctos, más la lógica de umbral y de filtro de fruta. Así pruebo la lógica de clasificación sin depender de los pesos del modelo.

P: ¿Qué rendimiento tiene bajo carga?

R: La inferencia toma ~200 ms; extremo a extremo desde Chile a AWS, ~600-700 ms con la red. Bajo concurrencia no hubo errores hasta 50 usuarios simultáneos; el cuello de botella es la CPU porque el modelo corre sin GPU. Lo medí con un script propio en un ambiente controlado.

P: ¿Qué falta o qué mejoraría?

R: Desplegar el backend de forma permanente (el lab AWS es temporal), automatizar las pruebas de extremo a extremo —hoy la cámara se prueba manualmente— y subir la cobertura.

P: ¿Por qué Android nativo y no multiplataforma?

R: Para aprovechar CameraX y el control fino de cámara y rendimiento; el alcance del proyecto es Android.

Tips: respira en los puntos · mira al jurado en las secciones [CLAVE] · si no sabes algo, reconócelo y explica cómo lo averiguarías · habla de decisiones y porqués, no solo de lo que hiciste.